

Below is a **version-by-version Python feature timeline from Python 1.0 through Python 3.14**. I have treated “Python version” as the **major/minor feature releases** rather than every maintenance release such as 3.13.1, 3.13.2, etc. Python.org’s release archive confirms the historical release sequence and dates. ([Python.org](https://python.org))

Note: “UniqueClassName” is a **suggested descriptive Python class name**, not an official class introduced by that Python release.

Python Version	Year	Major Features / Important Changes	UniqueClassName
Python 1.0	1994	Early stable Python; functions, modules, classes, exceptions, lists, tuples, dictionaries; lambda, map(), filter(), reduce(); object-oriented and functional programming foundations	Python10Core
Python 1.1	1994	lambda/functional-programming improvements; apply(), map(), filter(), reduce(); tuple(); type objects; types module; random, signal, traceback, urllib, mailbox, pty, tty modules	Python11Functional
Python 1.2	1995	Module/class/function docstrings; __doc__; enhanced import machinery; hierarchical imports; class exceptions; picklable classes; pickle, shelve, copy, os.path	Python12Modules
Python 1.3	1995	Keyword parameters/arguments ; third argument to raise; abstract-object interface; Tk improvements; new import hooks	Python13KeywordArgs
Python 1.4	1996	Complex numbers; new power operator behavior; improved slicing/indexing; class-private names using __name__; list(); __file__; improved standard library; early access-control/name-mangling mechanisms	Python14ComplexNumbers
Python 1.5	1998	assert statement; AssertionError; hierarchical package/import support; re regular-expression module; raw strings; class-based exceptions; dictionary methods such as get(), update(), copy(), clear(); improved embedding/threading	Python15Packages
Python 1.6	2000	Unicode support ; Unicode object; codecs/encodings; new SRE regular-expression engine; string methods; distutils; mmap;	Python16Unicode

		zipfile; urllib2; unicodedata; filecmp; UnboundLocalError; expanded C/API support	
Python 2.0	2000	Unicode strings ; list comprehensions; cyclic garbage collection; string methods; augmented assignment; XML support; gettext; codecs; encodings; distutils; improved regex engine; new development process based on PEPs	Python20Unicode
Python 2.1	2001	Nested/static scopes ; <code>__future__</code> directives; augmented assignment and other 2.0 features matured; improved warnings; function attributes; weak references; metadata/package improvements	Python21NestedScopes
Python 2.2	2001	New-style classes ; unified type/class model; iterators; generators; nested scopes enabled by default; unified int/long division; true division preparation; Unicode improvements	Python22NewStyleClasses
Python 2.3	2003	Built-in/standard set support via sets; generators finalized; <code>yield</code> ; <code>enumerate()</code> ; <code>sum()</code> ; Boolean type; <code>heapq</code> ; <code>datetime</code> ; logging; ZIP imports; <code>itertools</code> ; universal newline support	Python23Generators
Python 2.4	2004	Built-in set/frozenset; function/method decorators ; generator expressions; <code>decimal</code> ; <code>subprocess</code> ; <code>doctest</code> ; <code>functools</code> ; <code>collections.deque</code> ; conditional improvements; faster sorting	Python24Decorators
Python 2.5	2006	Conditional expressions (<code>x if condition else y</code>); <code>with</code> statement; context-management protocol; enhanced generators with <code>send()/throw()/close()</code> ; absolute/relative imports; <code>ctypes</code> ; <code>sqlite3</code> ; <code>ElementTree</code> ; <code>cProfile</code>	Python25ContextManager
Python 2.6	2008	Major Python 3 migration bridge ; <code>multiprocessing</code> ; <code>json</code> ; Abstract Base Classes (<code>abc</code>); improved string formatting; collections enhancements; warnings for Python 3 incompatibilities; <code>__future__</code> features	Python26MigrationBridge

Python 2.7	2010	Final 2.x feature release; OrderedDict; Counter; argparse; set/dict comprehensions; set literals; multiple context managers; memoryview; improved unittest; Python 3 compatibility features; improved io	Python27Legacy
Python 3.0	2008	Major incompatible Python 3 transition ; print() function; Unicode-first str; bytes; bytearray; true division; range() as lazy sequence; iterator-based map()/filter()/zip(); nonlocal; set literals/comprehensions; dict comprehensions; extended unpacking; function annotations; keyword-only arguments; new exception model	Python30ModernCore
Python 3.1	2009	collections.OrderedDict; thousands separator in formatting; improved io; improved importlib; concurrent.futures groundwork; better integer/string conversions; performance improvements	Python31OrderedDict
Python 3.2	2011	argparse; concurrent.futures; functools.lru_cache; int improvements; stable C ABI ; improved email; better SSL/security; dictionary-based logging configuration; improved multiprocessing	Python32Concurrency
Python 3.3	2012	yield from ; venv; asyncio foundations later developed; faulthandler; ipaddress; lzma; unittest.mock; importlib-based import machinery; flexible string/unicode handling; namespace packages	Python33YieldFrom
Python 3.4	2014	asyncio ; enum; pathlib; statistics; selectors; tracemalloc; ensurepip; functools singledispatch; improved SSL/TLS; safe object finalization; configurable memory allocators	Python34AsyncIO
Python 3.5	2015	async/await syntax ; coroutines; @ matrix multiplication operator; type hints/typing; generalized unpacking; bytes.hex(); zipapp; os.scandir(); asynchronous iteration foundations	Python35AsyncAwait

Python 3.6	2016	f-strings ; variable annotations; asynchronous generators; asynchronous comprehensions; numeric literal underscores; secrets; dataclasses groundwork; improved dictionaries; <code>__init_subclass__</code> preparation; faster CPython internals	Python36FStrings
Python 3.7	2018	Data classes ; <code>breakpoint()</code> ; <code>contextvars</code> ; postponed evaluation of annotations; official dictionary insertion-order guarantee; <code>async/await</code> become reserved keywords; <code>importlib.resources</code> ; typing improvements	Python37DataClasses
Python 3.8	2019	Assignment expressions <code>:=</code> (walrus operator); positional-only parameters <code>/</code> ; f-string = debugging; pickle protocol 5; runtime audit hooks; <code>vectorcall</code> ; improved initialization API; <code>TypedDict</code> ; <code>final</code> ; Literal typing	Python38WalrusOperator
Python 3.9	2020	Built-in collection generic type hints such as <code>list[int]</code> ; dictionary union <code> </code>	and
Python 3.10	2021	Structural pattern matching : <code>match/case</code> ; union types using <code>X Y</code> ; parenthesized context managers; improved error messages; precise line numbers; <code>zip(strict=True)</code> ; parameter specification/type improvements; <code>TypeAlias</code>	Python310PatternMatching
Python 3.11	2022	Exception Groups and <code>except*</code> ; <code>Exception.add_note()</code> ; <code>tomllib</code> ; fine-grained traceback locations; <code>Self</code> type; variadic generics; <code>TypedDict</code> required/not-required; major CPython performance improvements ; faster frames and startup	Python311ExceptionGroups
Python 3.12	2023	New type-parameter syntax ; type statement; formalized f-string grammar; per-interpreter GIL; low-impact monitoring; Python-level buffer protocol; comprehension inlining; <code>typing.override</code> ; improved <code>TypedDict</code> <code>**kwargs</code> ; <code>distutils</code> removed	Python312TypeParameters
Python 3.13	2024	Experimental free-threaded CPython / optional GIL removal;	Python313FreeThreading

		experimental JIT compiler; redesigned interactive interpreter; colored tracebacks; improved error messages; locals() semantics; type-parameter defaults; improved standard library; copy.replace()	
Python 3.14	2025	Template string literals / t-strings ; deferred annotation evaluation; multiple interpreters in standard library; officially supported free-threaded Python; Zstandard compression.zstd; improved asyncio introspection; control-flow changes in finally; safe external debugger interface; incremental garbage collection; experimental JIT in official Windows/macOS binaries	Python314TemplateStrings

Particularly important feature milestones

Feature	Introduced
lambda, map, filter, reduce	Python 1.0
Keyword arguments	Python 1.3
Complex numbers	Python 1.4
assert	Python 1.5
Packages	Python 1.5
Unicode	Python 1.6 / expanded in 2.0
List comprehensions	Python 2.0
Cyclic garbage collection	Python 2.0

Nested scopes	Python 2.1/2.2
New-style classes	Python 2.2
Generators / yield	Python 2.2
enumerate()	Python 2.3
set / frozenset	Python 2.4
Decorators	Python 2.4
Generator expressions	Python 2.4
with	Python 2.5
Conditional expressions	Python 2.5
multiprocessing	Python 2.6
OrderedDict	Python 2.7
Counter	Python 2.7
print() function	Python 3.0
bytes / Unicode str model	Python 3.0
nonlocal	Python 3.0
Dictionary/set comprehensions	Python 3.0
Function annotations	Python 3.0
yield from	Python 3.3

venv	Python 3.3
asyncio	Python 3.4
pathlib	Python 3.4
enum	Python 3.4
async / await	Python 3.5
Type hints / typing	Python 3.5
f-strings	Python 3.6
Variable annotations	Python 3.6
Data classes	Python 3.7
breakpoint()	Python 3.7
Walrus operator :=	Python 3.8
Positional-only /	Python 3.8
PEG parser	Python 3.9
Built-in generic types list[int]	Python 3.9
Dictionary union `	`
match / case	Python 3.10
Exception Groups / except*	Python 3.11
tomllib	Python 3.11

New type-parameter syntax	Python 3.12
type statement	Python 3.12
Experimental free-threading	Python 3.13
Experimental JIT	Python 3.13
Template strings / t-strings	Python 3.14
Deferred annotation evaluation	Python 3.14
Standard-library multiple interpreters	Python 3.14
Zstandard compression.zstd	Python 3.14

Python's official documentation describes each feature release through its **“What’s New”** series; these documents distinguish language syntax, built-ins, standard-library additions, interpreter/CPython changes, C-API changes, optimizations, deprecations, and removals. ([Python documentation](#))

For example, Python 3.14 was released on **7 October 2025** and its major changes include template strings, deferred annotations, multiple interpreters, free-threaded support, and Zstandard support. ([Python documentation](#)) Python 3.13 introduced experimental free-threading and a JIT, while Python 3.12 introduced the new type-parameter syntax and type statement. ([Python documentation](#))

Important: An absolutely exhaustive “all features” list—including every new standard-library function/class, C API change, bytecode change, optimization, deprecation and removal—would be **far larger than this table**. The table above is the consolidated **major-feature timeline** suitable for learning, interview preparation, documentation, or a Python-version comparison.

Here is the **Python major/minor version timeline from Python 1.0 through Python 3.14**, excluding maintenance releases such as 3.11.1, 3.11.2, etc.

#	Python Version	Release Year	Release Date	Status / Notes
1	Python 1.0	1994	Jan 1994	First major public release
2	Python 1.1	1994	Oct 1994	Early language/library expansion
3	Python 1.2	1995	Apr 1995	Module and object-system improvements
4	Python 1.3	1995	Oct 1995	Keyword arguments introduced
5	Python 1.4	1996	Oct 1996	Language and standard-library expansion
6	Python 1.5	1998	Jan 1998	Packages, assertions, improved exceptions
7	Python 1.6	2000	Sep 2000	Unicode and major library improvements
8	Python 2.0	2000	Oct 2000	Major 2.x release; list comprehensions, Unicode, GC
9	Python 2.1	2001	Apr 2001	Nested scopes and <code>_future_</code>
10	Python 2.2	2001	Dec 2001	New-style classes, iterators, generators
11	Python 2.3	2003	Jul 2003	Generators, Boolean type, datetime, itertools
12	Python 2.4	2004	Nov 2004	Decorators, generator expressions, built-in sets
13	Python 2.5	2006	Sep 2006	with, conditional expressions, enhanced generators
14	Python 2.6	2008	Oct 2008	Python 3 migration/compatibility release
15	Python 2.7	2010	Jul 2010	Final Python 2 feature release

16	Python 3.0	2008	Dec 2008	Major Python 3 redesign
17	Python 3.1	2009	Jun 2009	OrderedDict, formatting improvements
18	Python 3.2	2011	Feb 2011	argparse, concurrent.futures, lru_cache
19	Python 3.3	2012	Sep 2012	yield from, venv, namespace packages
20	Python 3.4	2014	Mar 2014	asyncio, pathlib, enum, statistics
21	Python 3.5	2015	Sep 2015	async/await, type hints, @ operator
22	Python 3.6	2016	Dec 2016	f-strings, variable annotations, async generators
23	Python 3.7	2018	Jun 2018	Data classes, breakpoint(), contextvars
24	Python 3.8	2019	Oct 2019	Walrus operator :=, positional-only parameters
25	Python 3.9	2020	Oct 2020	PEG parser, built-in generic types, dict union
26	Python 3.10	2021	Oct 2021	Structural pattern matching match/case
27	Python 3.11	2022	Oct 2022	Exception Groups, except*, major performance improvements
28	Python 3.12	2023	Oct 2023	New type-parameter syntax, type statement
29	Python 3.13	2024	Oct 2024	Free-threaded CPython experiment, experimental JIT
30	Python 3.14	2025	Oct 2025	Template strings, deferred annotations, multiple interpreters

Version-family summary

Python Family	Versions	Period	Significance
Python 1.x	1.0–1.6	1994–2000	Original Python language evolution
Python 2.x	2.0–2.7	2000–2010	Mature Python 2 generation
Python 3.x	3.0–3.14	2008–2025	Modern Python generation
Current 3.x line covered here	3.14	2025	Latest version in your requested range

Total: 30 major/minor feature releases from **Python 1.0** through **Python 3.14**.

Note that **Python 3.0 was released before Python 2.7**; the two development lines overlapped for several years. Python 2.7 was subsequently the final Python 2 feature release.

Python terminology needs one distinction: “**built-in libraries**” is not the same as “**standard library**.” Python has built-in functions/classes available without import, while the **standard library** is the much larger collection of modules/packages distributed with Python. The exact set also varies by Python version and operating system. ([Python documentation](#))

Because you asked for **Python 1.0–3.14**, an exhaustive version-by-version list would contain hundreds of historical modules, including modules that were later renamed, deprecated, or removed. The table below therefore gives the **major standard-library modules/packages that developers commonly use**, with their practical purpose and examples. Python 3.14's official module index documents the current complete module inventory. ([Python documentation](#))

Python standard-library modules: practical reference

#	Library / Module	Main purpose	When to use	When NOT to use	Real-time use case	Example
1	builtins	Built-in functions/classes	Access or wrap built-ins programmatically	Don't import unnecessarily	Custom wrapper around open()	builtins.open("a.txt")
2	sys	Python runtime/system information	CLI args, paths, interpreter info, exit codes	Don't use for normal application configuration	Read command-line arguments	sys.argv
3	os	OS interaction	Environment variables, processes, filesystem	Don't use when a higher-level module is clearer	Read HOME/PATH	os.getenv("HOME")
4	os.path	Path manipulation	Legacy path operations	Prefer pathlib for new code	Build file paths	os.path.join("data", "a.csv")
5	pathlib	Object-oriented paths	Modern filesystem applications	Not necessary for tiny string-only operations	Find application files	Path("data").glob("*.csv")

6	shutil	High-level file operations	Copy/move/archive files	Don't use for simple file reading	Backup directory	shutil.copytree("src","backu p")
7	glob	Filename pattern matching	Find files by wildcard	Don't use for complex filesystem traversal	Find all PDFs	glob.glob("*.pdf")
8	fnmatch	Filename matching	Match names using shell-style patterns	Don't use for full regex requirements	Filter .log files	fnmatch.fnmatch(name,"*.lo g")
9	tempfile	Temporary files/directories	Intermediate processing	Don't store permanent data there	Generate temporary upload file	tempfile.NamedTemporary File()
10	filecmp	Compare files/directories	Backup validation	Don't use for semantic/content analysis	Compare two folders	filecmp.cmp("a.txt","b.txt")
11	stat	File metadata/modes	Detailed filesystem inspection	Not needed for ordinary file access	Check permissions	stat.S_ISREG(mode)
12	io	In-memory/file-like I/O	Streams, BytesIO, StringIO	Don't use when normal file APIs suffice	Process uploaded bytes	io.BytesIO(data)
13	pickle	Python object serialization	Trusted Python-only persistence	Never unpickle untrusted data	Cache Python objects	pickle.dump(obj,f)
14	json	JSON serialization	APIs/config/data exchange	Don't use for binary/high-performance serialization	REST API payload	json.dumps(data)

15	csv	CSV files	Spreadsheet/tabular exchange	Don't use for complex databases	Import sales data	csv.DictReader(file)
16	sqlite3	Embedded SQL database	Local applications/prototypes	Don't use for high-scale distributed DB workloads	Desktop app database	sqlite3.connect("app.db")
17	dbm	Persistent key-value storage	Simple local key/value data	Don't use for relational queries	Small local cache	db["user"]="Anurag"
18	configparser	INI configuration	Legacy/simple configuration	Prefer structured config for complex systems	Application settings	config["server"]["port"]
19	tomllib	TOML parsing	Modern configuration files	Don't use for JSON/YAML data	Read pyproject.toml	tomllib.loads(text)
20	re	Regular expressions	Pattern matching/extraction	Don't use regex for complex parsing	Extract emails	re.findall(pattern,text)
21	string	String constants/utilities	Templates/character sets	Don't use when normal string methods suffice	Generate character sets	string.ascii_letters
22	textwrap	Format text	CLI/report formatting	Not needed for simple strings	Wrap terminal output	textwrap.fill(text,80)
23	difflib	Compare text/sequences	Diff/search/similarity	Don't use as a full NLP engine	Compare two files	difflib.unified_diff(a,b)
24	unicodedata	Unicode information	International text processing	Not needed for plain ASCII	Normalize user names	unicodedata.normalize(...)

25	math	Mathematical functions	Numeric calculations	Don't use for arbitrary-precision decimal finance	Engineering calculation	math.sqrt(25)
26	cmath	Complex mathematics	Complex-number calculations	Don't use for ordinary real numbers	Signal calculations	cmath.sqrt(-1)
27	decimal	Decimal arithmetic	Financial/accounting calculations	Don't use when binary floating point is sufficient	Invoice totals	Decimal("10.25")
28	fractions	Rational numbers	Exact fractions	Don't use when floats are adequate	Exact ratio calculation	Fraction(1,3)
29	random	Pseudo-random values	Simulation/testing/non-security randomness	Not for passwords/tokens	Random test data	random.randint(1,100)
30	secrets	Cryptographically secure randomness	Password/token generation	Don't use merely for simple simulations	Password-reset token	secrets.token_urlsafe()
31	statistics	Statistics	Mean/median/std dev	Don't use for large-scale scientific analytics	Calculate average latency	statistics.mean(values)
32	numbers	Numeric ABCs	Generic numeric APIs	Usually unnecessary in application code	Accept different numeric types	isinstance(x,numbers.Real)

33	datetime	Dates/times	Business/application timestamps	Don't use naive datetimes for timezone-sensitive systems	Calculate invoice due date	datetime.now()
34	zoneinfo	IANA time zones	Correct timezone handling	Don't manually hard-code offsets	Convert meeting times	ZoneInfo("Asia/Kolkata")
35	calendar	Calendar calculations	Month/week/date calculations	Don't use for timezone conversion	Generate monthly calendar	calendar.month(2026,9)
36	time	Low-level time functions	Timers, timestamps, sleeping	Don't use for rich date calculations	Measure elapsed time	time.monotonic()
37	timeit	Benchmark small code	Performance testing	Don't use as application timer	Compare two algorithms	timeit.timeit(...)
38	collections	Specialized containers	Counter, deque, defaultdict	Don't use if ordinary collections suffice	Count words	Counter(words)
39	collections.abc	Collection interfaces	Type/interface checking	Not needed for normal iteration	Validate iterable API	isinstance(x,Iterable)
40	heapq	Priority queues/heaps	Scheduling/top-N algorithms	Don't use for ordinary sorting	Job priority queue	heapq.heappush(q,item)
41	bisect	Sorted-list insertion/search	Maintain sorted lists	Don't use for large database-scale data	Insert ranked score	bisect.insort(scores,90)

42	array	Compact typed arrays	Memory-efficient numeric arrays	Prefer NumPy for serious numerical work	Store sensor readings	<code>array("d",[1.2,2.3])</code>
43	weakref	Weak references	Cache/object lifecycle management	Don't use for ordinary references	Avoid cache memory leaks	<code>weakref.ref(obj)</code>
44	copy	Copy objects	Shallow/deep copying	Don't blindly deep-copy huge objects	Clone configuration	<code>copy.deepcopy(config)</code>
45	pprint	Pretty-print structures	Debugging complex objects	Don't use for machine-readable output	Debug API response	<code>pprint(data)</code>
46	enum	Enumerations	Fixed named choices	Don't use for arbitrary dynamic values	Order/status types	<code>class Status(Enum): ...</code>
47	dataclasses	Data-focused classes	DTOs/config/result objects	Don't use when complex behavior dominates	API response model	<code>@dataclass class User:</code>
48	types	Runtime type utilities	Dynamic type inspection	Don't use instead of normal typing	Create namespace	<code>types.SimpleNamespace()</code>
49	typing	Type annotations	Static analysis/documentation	Doesn't enforce runtime types by itself	Type-safe service APIs	<code>list[str]</code>
50	abc	Abstract base classes	Define interfaces/contracts	Don't over-engineer tiny classes	Plugin architecture	<code>class Storage(ABC)</code>

51	functools	Functional utilities	Caching, decorators, partial functions	Don't complicate simple functions	Memoize expensive calculation	@functools.lru_cache
52	itertools	Iterator algorithms	Efficient large-data iteration	Don't use if a simple loop is clearer	Process combinations	itertools.combinations()
53	operator	Operators as functions	Functional programming/sorting	Don't use when operators are clearer	Sort by field	operator.itemgetter(1)
54	contextlib	Context managers	Resource management	Don't use for resources already context-managed	Custom transaction context	@contextmanager
55	dataclasses	Structured data objects	Lightweight models	Don't use as a replacement for ORM models	Configuration DTO	@dataclass
56	logging	Application logging	Production diagnostics	Don't use print() for production logging	API request logs	logging.info("Started")
57	warnings	Warning management	Deprecation/runtime warnings	Don't hide important warnings globally	Warn about deprecated API	warnings.warn(...)
58	traceback	Exception traceback handling	Error reporting	Don't expose sensitive tracebacks to users	Error logging	traceback.format_exc()

59	pdb	Debugger	Interactive debugging	Don't leave debugger calls in production	Debug failing function	breakpoint()
60	inspect	Runtime introspection	Analyze functions/classes	Don't depend heavily on implementation details	Inspect function signature	inspect.signature(fn)
61	dis	Bytecode inspection	Learning/perform ance investigation	Not normal application logic	Inspect compiled function	dis.dis(fn)
62	ast	Python syntax trees	Static analysis/code transformation	Don't parse simple text with AST	Analyze Python source	ast.parse(code)
63	compile	Compile source/code	Dynamic compilation/tooling	Never compile untrusted code	Build code-analysis tool	compile(src,"x","exec")
64	unittest	Unit testing	Standard Python tests	Don't need it if project standardizes another framework	Test service logic	unittest.TestCase
65	unittest.mock	Mock dependencies	Unit-test isolation	Don't mock everything	Mock external API	Mock(return_value=200)
66	doctest	Test documentation examples	Small examples/API docs	Not ideal for complex test suites	Test tutorial examples	doctest.testmod()

67	venv	Virtual environments	Isolate project dependencies	Don't create one for every command	Isolate Flask project	<code>python -m venv .venv</code>
68	ensurepip	Bootstrap pip	Install pip into Python environment	Usually unnecessary when pip is already installed	Repair minimal installation	<code>python -m ensurepip</code>
69	zipapp	Executable Python ZIP apps	Package CLI applications	Don't use for complex deployment	Distribute CLI tool	<code>python -m zipapp app</code>
70	zipfile	ZIP archives	Compress/extract files	Don't use for high-performance compression pipelines	Create deployment package	<code>ZipFile("app.zip")</code>
71	tarfile	TAR archives	Linux/server backups	Don't use for ZIP-specific interoperability	Create .tar.gz backup	<code>tarfile.open(...)</code>
72	gzip	GZIP compression	Compress streams/files	Don't use when random-access archive needed	Compress logs	<code>gzip.open("app.log.gz")</code>
73	bz2	BZIP2 compression	High compression archives	Don't use if compatibility/performance requires gzip	Compress archival data	<code>bz2.open(...)</code>
74	lzma	LZMA/XZ compression	High-ratio compression	Don't use when CPU/latency is critical	Compress large backups	<code>lzma.open(...)</code>

75	base64	Binary-to-text encoding	API/email/file transport	Not encryption	Encode binary payload	base64.b64encode(data)
76	hashlib	Cryptographic hashes	Checksums/password hashing primitives	Don't use raw fast hashes for password storage	File integrity check	hashlib.sha256(data)
77	hmac	Message authentication	API signature verification	Don't use as general encryption	Verify webhook signature	hmac.compare_digest(...)
78	secrets	Secure tokens	Authentication/security tokens	Not for deterministic testing	Session token	secrets.token_hex()
79	ssl	TLS/SSL	Secure network connections	Don't implement TLS yourself	Secure socket/HTTPS	ssl.create_default_context()
80	socket	Low-level networking	TCP/UDP/network programming	Don't use when HTTP client is sufficient	TCP server	socket.socket()
81	socketserver	Network server framework	Simple TCP/UDP servers	Don't use for modern high-scale web APIs	Internal TCP server	socketserver.TCPServer
82	selectors	I/O multiplexing	High-performance socket I/O	Don't use for ordinary scripts	Event-driven server	selectors.DefaultSelector()
83	asyncio	Asynchronous I/O	High-concurrency I/O applications	Don't use if workload is mostly CPU-bound	Async API client	asyncio.run(main())

84	threading	Threads	I/O-bound concurrent work	Don't expect it to speed CPU-heavy Python code under normal GIL builds	Concurrent file downloads	threading.Thread(...)
85	multiprocessing	Process-based parallelism	CPU-bound workloads	Don't use when lightweight async I/O is enough	CPU-intensive image processing	multiprocessing.Pool()
86	concurrent.futures	Thread/process pools	Simple concurrent task execution	Don't use if direct async design is better	Parallel API calls	ThreadPoolExecutor()
87	queue	Thread-safe queues	Producer/consumer systems	Don't use as a general database	Worker job queue	queue.Queue()
88	subprocess	Run external programs	Integrate OS commands	Never pass unsafe user input directly to shell	Run Git command	subprocess.run(["git", "status"])
89	signal	OS signal handling	Graceful shutdown	Platform-specific signal behavior matters	Handle SIGTERM	signal.signal(...)
90	mmap	Memory-mapped files	Large-file access	Don't use for ordinary small files	Process huge binary file	mmap.mmap(...)
91	http.client	Low-level HTTP	Custom HTTP clients	Prefer higher-level libraries for most apps	Direct HTTP protocol work	HTTPConnection(...)
92	http.server	Basic HTTP server	Development/testing/internal tools	Not a production web server	Serve local files	python -m http.server

93	urllib.request	HTTP/URL access	Simple HTTP requests without dependencies	Don't use when advanced HTTP client features are required	Download a file	urllib.request.urlopen(url)
94	urllib.parse	Parse URLs	Query/path manipulation	Don't manually split URLs	Parse query parameters	urlparse(url)
95	urllib.robotparser	Robots.txt handling	Respect website crawling rules	Don't assume it guarantees legal access	Crawler policy check	RobotFileParser()
96	email	Email message handling	Parse/create MIME messages	Not an SMTP transport itself	Parse received email	email.message_from_bytes(..)
97	smtplib	SMTP email sending	Send email through SMTP	Don't use if provider API is required	Send notification	smtplib.SMTP(...)
98	imaplib	IMAP email access	Read/manage mailboxes	Don't use for POP-only servers	Read inbox	imaplib.IMAP4_SSL(...)
99	poplib	POP3 email access	Download email from POP3	Don't use when mailbox synchronization needs IMAP	Download messages	poplib.POP3_SSL(...)
100	ftplib	FTP	Legacy FTP systems	Don't use insecure FTP for sensitive data	Transfer legacy files	ftplib.FTP(...)
101	uuid	Unique identifiers	IDs for records/resources	Don't use as a security token by itself	Generate request ID	uuid.uuid4()

102	ipaddress	IPv4/IPv6 operations	Network validation/calculation	Don't manually parse IP strings	Validate client IP	ipaddress.ip_address(ip)
103	xml.etree.ElementTree	XML parsing	Simple XML documents	Don't use for highly complex XML requirements	Parse XML feed	ET.fromstring(xml)
104	xml.dom	DOM-style XML	DOM XML processing	Prefer ElementTree for simple XML	Manipulate XML document	xml.dom.minidom.parse()
105	xml.sax	Streaming XML parsing	Large XML files	Don't use if document-tree manipulation is needed	Parse huge XML feed	xml.sax.parse(...)
106	html	HTML utilities	Escape/unescape HTML	Not an HTML web framework	Escape user text	html.escape(text)
107	html.parser	Basic HTML parsing	Lightweight HTML parsing	Don't use as a full browser/parser	Extract links	HTMLParser()
108	sqlite3	Local relational DB	Embedded applications	Not ideal for distributed production DB	Desktop inventory app	conn.execute(...)
109	curses	Terminal UI	Unix terminal applications	Don't use for GUI/web apps	Terminal dashboard	curses.wrapper(main)
110	tkinter	Desktop GUI	Simple cross-platform GUI	Don't use for modern complex UI requirements	Desktop calculator	tkinter.Tk()

111	turtle	Educational graphics	Learning Python/geometry	Not for production graphics	Draw shapes	<code>turtle.forward(100)</code>
112	gettext	Internationalization	Translate application text	Don't use for one-language apps	Multilingual desktop app	<code>gettext.gettext("Hello")</code>
113	locale	Locale-specific behavior	Formatting/national conventions	Don't use as a substitute for timezone handling	Number formatting	<code>locale.setlocale(...)</code>
114	argparse	CLI argument parsing	Production command-line tools	Don't use for one-option trivial scripts	Build CLI utility	<code>argparse.ArgumentParser()</code>
115	getopt	Basic CLI parsing	Legacy/simple command-line interfaces	Prefer argparse for new projects	Parse -v option	<code>getopt.getopt(...)</code>
116	readline	Interactive terminal editing	CLI shells	Platform support varies	Command history	<code>readline.readline()</code>
117	cmd	Command interpreter	Build interactive CLI shells	Don't use for ordinary CLIs	Database admin shell	<code>class Shell(cmd.Cmd)</code>
118	logging.config	Logging configuration	Centralized logging setup	Don't need it for tiny scripts	Configure production logs	<code>dictConfig(config)</code>
119	logging.handlers	Specialized log handlers	Rotating/syslog/network logs	Don't use unnecessarily	Rotate application logs	<code>RotatingFileHandler(...)</code>
120	contextvars	Context-local state	Async applications/request context	Don't use as global state	Request correlation ID	<code>ContextVar("request_id")</code>

121	atexit	Program cleanup	Cleanup at normal interpreter exit	Not reliable for crashes/forced termination	Flush local cache	atexit.register(cleanup)
122	warnings	Runtime warnings	Deprecation notices	Don't suppress warnings blindly	API deprecation notice	warnings.warn(...)
123	resource	Unix resource limits	Process resource monitoring	Unix-specific	Monitor CPU/memory limits	resource.getrusage(...)
124	platform	OS/platform information	Cross-platform diagnostics	Don't use to replace feature detection	Detect OS version	platform.system()
125	sysconfig	Python installation configuration	Build/install tooling	Not normal application logic	Find Python paths	sysconfig.get_paths()
126	importlib	Import machinery	Dynamic/plugin loading	Don't dynamically import without need	Plugin system	importlib.import_module(name)
127	pkgutil	Package/module discovery	Plugin/package discovery	Prefer modern importlib APIs where applicable	Discover modules	pkgutil.iter_modules()
128	pydoc	Documentation/help	Generate/read Python docs	Don't replace dedicated documentation systems	Generate module docs	pydoc.render_doc(...)
129	trace	Trace execution	Debugging/coverage-style analysis	Not for production monitoring	Trace code execution	trace.Trace(...)

130	tracemalloc	Memory allocation tracing	Find Python memory issues	Don't leave enabled unnecessarily in production	Find memory growth	tracemalloc.start()
131	profile	Profiling	CPU performance analysis	Don't use continuously in production	Find slow functions	profile.run(...)
132	cProfile	Deterministic profiling	Application performance analysis	Don't profile every request in production	Profile API endpoint	cProfile.run(...)
133	resource	Resource usage	Unix process metrics	Not portable across all OSes	CPU/resource diagnostics	resource.getrusage(...)
134	graphlib	Graph algorithms	Dependency ordering	Don't use for ordinary lists	Package dependency resolution	TopologicalSorter(...)
135	pdb	Debugging	Step-through debugging	Don't use as production monitoring	Investigate exception	pdb.set_trace()
136	zipimport	Import Python code from ZIP	Legacy/package internals	Don't use when normal packaging works	Import zipped application	zipimport.zipimporter(...)
137	compression.zstd	Zstandard compression	Modern high-performance compression	Don't use where older Python compatibility is required	Compress large data	compression.zstd...
138	annotationlib	Annotation introspection	Advanced type/annotation tooling	Not needed for ordinary type hints	Framework annotation processing	annotationlib...

Historical modules

Because your range starts at **Python 1.0**, some libraries that existed historically are **not available in Python 3.14**. Examples include modules that were renamed, replaced, deprecated, or removed. Python's current module index explicitly marks some historical modules as removed—for example, `aifc` and `audioop` are removed in Python 3.13, while `asynchat` and `asyncore` were removed earlier. ([Python documentation](#))

Historical library	Historical role	Modern direction
urllib	Early URL handling	<code>urllib.request</code> , <code>urllib.parse</code>
htmllib	Old HTML parsing	<code>html.parser</code>
sgmlib	SGML parsing	Modern XML/HTML tools
cStringIO	Faster string I/O	<code>io.BytesIO</code> / <code>io.StringIO</code>
StringIO	In-memory text I/O	<code>io.StringIO</code>
sets	Set implementation	Built-in <code>set</code> / <code>frozenset</code>
sets.Set	Set class	Built-in <code>set</code>
md5	MD5 hashing	<code>hashlib</code>
sha	SHA hashing	<code>hashlib</code>
ConfigParser	Configuration parsing	<code>configparser</code>
Queue	Queue implementation	<code>queue</code>
SimpleHTTPServer	Basic HTTP server	<code>http.server</code>
BaseHTTPServer	HTTP server framework	<code>http.server</code>

SocketServer	Socket server framework	socketserver
Tkinter	GUI	tkinter
cPickle	Fast pickle implementation	pickle
imp	Import machinery	importlib
optparse	CLI parsing	argparse
distutils	Package/build utilities	setuptools / modern packaging tools
asyncore	Asynchronous sockets	asyncio
asynchat	Async socket chat	asyncio
cgi	CGI support	Web frameworks / WSGI/ASGI
telnetlib	Telnet client	Third-party/network-specific alternatives
aifc	AIFF audio files	Third-party audio libraries
audioop	Raw audio operations	Third-party audio/numerical libraries

How to think about the standard library

A useful practical classification is:

Category	Important libraries	Typical applications
Core/System	sys, os, platform, subprocess	OS automation, CLI tools

Files	pathlib, shutil, glob, tempfile	File management
Data	json, csv, pickle, sqlite3	Data storage/exchange
Text	re, string, textwrap, difflib	Parsing/search/reporting
Math	math, decimal, fractions, statistics	Calculations/finance
Collections	collections, heapq, bisect, array	Algorithms/data structures
Date/time	datetime, zoneinfo, calendar, time	Scheduling/business systems
Concurrency	threading, multiprocessing, asyncio, concurrent.futures	Parallel/concurrent applications
Networking	socket, ssl, urllib, http, ipaddress	Network applications
Email	email, smtplib, imaplib, poplib	Email automation
Security	hashlib, hmac, secrets, ssl	Authentication/integrity/TLS
Testing	unittest, doctest, unittest.mock	Automated testing
Debugging	pdb, traceback, tracemalloc, inspect	Debugging/performance
Packaging	venv, ensurepip, zipapp, importlib	Deployment/environment management
GUI	tkinter, turtle, curses	Desktop/terminal applications
XML/HTML	xml, html, html.parser	Document processing
Compression	zipfile, tarfile, gzip, bz2, lzma, compression.zstd	Backups/data transfer
Internationalization	gettext, locale, unicodedata	Multilingual applications

The official Python documentation makes the same fundamental distinction: the standard library contains both **built-in modules written in C** and modules written in Python, while the separate Built-ins Reference covers functions, classes, types, and exceptions available without an import. ([Python documentation](#))

Important for your Python 1.0 → 3.14 requirement: a truly exhaustive table would need to be split into **historical version-specific tables**, because “all libraries available in Python 1.0,” “all libraries added in Python 2.x,” and “all libraries in Python 3.14” are different sets. The current 3.14 module index alone is extensive and includes both current modules and entries marked deprecated/removed. ([Python documentation](#))

Yes. Below is a **practical code-reference for every individual library/module listed above**. The examples are intentionally small so you can copy them into a Python project and experiment with them.

Important: Python 1.0–3.14 spans more than 30 years. Some modules below are modern modules, some existed under older names, and some historical modules have been removed. Therefore, an example marked **Python 3.x** should not be expected to run on Python 1.x/2.x. Likewise, some old modules require the corresponding historical Python version.

1. builtins

Built-in functions and types are available without importing anything.

```
name = "Anurag"
numbers = [10, 20, 30]

print(name)
print(len(numbers))
print(max(numbers))
```

Use for fundamental Python operations.

2. sys

Interact with the Python interpreter.

```
import sys

print("Python version:", sys.version)
print("Arguments:", sys.argv)
```

```
if len(sys.argv) < 2:  
    sys.exit("Please provide a name")  
  
print("Hello", sys.argv[1])
```

Typical use: CLI applications, interpreter information, exit codes.

3. os

Interact with the operating system.

```
import os  
  
print("Current directory:", os.getcwd())  
  
os.makedirs("data", exist_ok=True)  
  
print("HOME:", os.environ.get("HOME"))
```

Typical use: environment variables, processes, OS-level operations.

4. os.path

Manipulate filesystem paths.

```
import os  
  
path = os.path.join("data", "users.txt")
```

```
print(path)
print(os.path.exists(path))
print(os.path.basename(path))
print(os.path.dirname(path))
```

For new applications, pathlib is usually more convenient.

5. pathlib

Modern filesystem API.

```
from pathlib import Path

folder = Path("data")
folder.mkdir(exist_ok=True)

file = folder / "users.txt"
file.write_text("Anurag\nSakshi")

print(file.exists())
print(file.read_text())
```

Typical use: modern Python file handling.

6. shutil

Copy and move files/directories.

```
import shutil
```

```
shutil.copy("source.txt", "backup.txt")
```

Directory example:

```
shutil.copytree("project", "project_backup", dirs_exist_ok=True)
```

7. glob

Find files using patterns.

```
import glob

files = glob.glob("data/*.csv")

for file in files:
    print(file)
```

Useful for batch-processing files.

8. fnmatch

Match filenames.

```
import fnmatch

files = ["a.py", "b.txt", "test.py"]
```

```
for file in files:
    if fnmatch.fnmatch(file, "*.py"):
        print(file)
```

9. tempfile

Create temporary files.

```
import tempfile

with tempfile.NamedTemporaryFile(mode="w+", delete=False) as file:
    file.write("Temporary data")
    print(file.name)
```

Do not use temporary files for permanent application storage.

10. filecmp

Compare files.

```
import filecmp

same = filecmp.cmp("file1.txt", "file2.txt")

print("Same:", same)
```

11. stat

Inspect filesystem metadata.

```
import os
import stat

info = os.stat("file.txt")

print("Size:", info.st_size)
print("Regular file:", stat.S_ISREG(info.st_mode))
```

12. io

Work with in-memory streams.

```
import io

stream = io.StringIO()

stream.write("Hello ")
stream.write("Python")

print(stream.getvalue())
```

Binary:

```
binary = io.BytesIO(b"Hello")
print(binary.read())
```

13. pickle

Serialize Python objects.

```
import pickle

data = {
    "name": "Anurag",
    "age": 43
}

with open("data.pkl", "wb") as f:
    pickle.dump(data, f)

with open("data.pkl", "rb") as f:
    restored = pickle.load(f)

print(restored)
```

Never load untrusted pickle files.

14. json

JSON data exchange.

```
import json

user = {
    "name": "Anurag",
```

```
"skills": ["Python", "Java", "AWS"]
}
```

```
text = json.dumps(user, indent=2)
```

```
print(text)
```

```
data = json.loads(text)
```

```
print(data["name"])
```

Very common in REST APIs.

15. csv

Read CSV files.

```
import csv
```

```
with open("users.csv", newline="") as f:
    reader = csv.DictReader(f)
```

```
    for row in reader:
        print(row["name"])
```

Write CSV:

```
with open("users.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Age"])
    writer.writerow(["Anurag", 43])
```

16. sqlite3

Embedded relational database.

```
import sqlite3

connection = sqlite3.connect("users.db")

connection.execute(
    "CREATE TABLE IF NOT EXISTS users "
    "(id INTEGER PRIMARY KEY, name TEXT)"
)

connection.execute(
    "INSERT INTO users(name) VALUES (?)",
    ("Anurag",)
)

connection.commit()

rows = connection.execute("SELECT * FROM users")

for row in rows:
    print(row)

connection.close()
```

Excellent for local applications and prototypes.

17. dbm

Simple persistent key-value storage.

```
import dbm

with dbm.open("users", "c") as db:
    db["name"] = "Anurag"
    print(db["name"].decode())
```

Use SQLite instead when you need relational queries.

18. configparser

Read INI configuration.

```
import configparser

config = configparser.ConfigParser()

config["server"] = {
    "host": "localhost",
    "port": "8080"
}

with open("config.ini", "w") as f:
    config.write(f)
```

Read:

```
config.read("config.ini")
```

```
print(config["server"]["port"])
```

19. tomlib

Python 3.11+.

Read TOML:

```
import tomlib

with open("config.toml", "rb") as f:
    config = tomlib.load(f)

print(config)
```

Useful for pyproject.toml and modern configuration.

20. re

Regular expressions.

```
import re

text = "Contact anurag@example.com"

emails = re.findall(
    r"[\w.+-]+@[ \w.-]+\.\w+",
    text
```

```
)  
print(emails)
```

Useful for pattern extraction and validation.

21. string

String constants and utilities.

```
import string  
  
print(string.ascii_letters)  
print(string.digits)  
print(string.punctuation)
```

22. textwrap

Format long text.

```
import textwrap  
  
text = "Python is a powerful programming language used in many industries."  
  
print(textwrap.fill(text, width=30))
```

23. difflib

Compare text.

```
import difflib

old = ["Python", "Java", "C++"]
new = ["Python", "Java", "Go"]

for line in difflib.unified_diff(old, new):
    print(line)
```

24. unicodedata

Unicode information.

```
import unicodedata

text = "é"

print(unicodedata.name(text))
print(unicodedata.category(text))
```

Normalization:

```
normalized = unicodedata.normalize("NFC", text)
```

25. math

Mathematical operations.

```
import math

print(math.sqrt(25))
print(math.pi)
print(math.sin(math.pi / 2))
```

26. cmath

Complex-number mathematics.

```
import cmath

result = cmath.sqrt(-1)

print(result)
```

Output:

```
1j
```

27. decimal

Accurate decimal arithmetic.

```
from decimal import Decimal
```

```
price = Decimal("10.10")
```

```
tax = Decimal("1.50")
```

```
total = price + tax
```

```
print(total)
```

Useful for financial calculations.

28. fractions

Exact rational arithmetic.

```
from fractions import Fraction
```

```
a = Fraction(1, 3)
```

```
b = Fraction(1, 6)
```

```
print(a + b)
```

Output:

```
1/2
```

29. random

Pseudo-random values.

```
import random

print(random.randint(1, 100))
print(random.choice(["Python", "Java", "C++"]))
```

Do not use for passwords/security tokens.

30. secrets

Cryptographically secure random values.

```
import secrets

token = secrets.token_urlsafe(32)

print(token)
```

Useful for password-reset/session tokens.

31. statistics

Basic statistics.

```
import statistics
```

```
values = [10, 20, 30, 40, 50]

print(statistics.mean(values))
print(statistics.median(values))
print(statistics.stdev(values))
```

32. numbers

Numeric type abstractions.

```
import numbers

value = 10

print(isinstance(value, numbers.Number))
print(isinstance(value, numbers.Integral))
```

33. datetime

Date and time.

```
from datetime import datetime, timedelta

now = datetime.now()

print(now)
```

```
tomorrow = now + timedelta(days=1)
print(tomorrow)
```

34. zoneinfo

Python 3.9+ timezone support.

```
from datetime import datetime
from zoneinfo import ZoneInfo

india_time = datetime.now(
    ZoneInfo("Asia/Kolkata")
)

print(india_time)
```

35. calendar

Calendar operations.

```
import calendar

print(calendar.month(2026, 9))
```

36. time

Low-level time functions.

```
import time
start = time.monotonic()
time.sleep(1)
elapsed = time.monotonic() - start
print("Elapsed:", elapsed)
```

37. timeit

Benchmark code.

```
import timeit
result = timeit.timeit(
    "sum(range(1000))",
    number=10000
)
print(result)
```

38. collections

Specialized containers.

```
from collections import Counter  
  
words = ["python", "java", "python", "go"]  
  
counts = Counter(words)  
  
print(counts)
```

Other important classes include:

```
deque  
defaultdict  
OrderedDict  
Counter  
ChainMap
```

39. collections.abc

Collection interfaces.

```
from collections.abc import Iterable  
  
data = [1, 2, 3]
```

```
print(isinstance(data, Iterable))
```

40. heapq

Priority queue.

```
import heapq

queue = []

heapq.heappush(queue, (2, "Normal"))
heapq.heappush(queue, (1, "Critical"))

print(heapq.heappop(queue))
```

Useful for job scheduling.

41. bisect

Maintain sorted lists.

```
import bisect

numbers = [10, 20, 30]

bisect.insort(numbers, 25)

print(numbers)
```

42. array

Compact typed arrays.

```
from array import array
numbers = array("i", [10, 20, 30])
numbers.append(40)
print(numbers)
```

For scientific computing, NumPy is usually more capable.

43. weakref

Weak references.

```
import weakref

class User:
    pass

user = User()
reference = weakref.ref(user)
print(reference())
```

```
del user  
print(reference())
```

Useful for cache/object lifecycle scenarios.

44. copy

Copy objects.

```
import copy  
  
original = {  
    "name": "Anurag",  
    "skills": ["Python", "Java"]  
}  
  
new_object = copy.deepcopy(original)  
  
new_object["skills"].append("AWS")  
  
print(original)  
print(new_object)
```

45. pprint

Pretty-print complex data.

```
from pprint import pprint

data = {
    "users": [
        {"name": "Anurag", "skills": ["Python", "Java"]},
        {"name": "Sakshi", "skills": ["Java"]}
    ]
}

pprint(data)
```

46. enum

Enumerated constants.

```
from enum import Enum

class Status(Enum):
    NEW = 1
    PROCESSING = 2
    COMPLETE = 3

print(Status.NEW)
```

Useful for fixed application states.

47. dataclasses

Python 3.7+.

```
from dataclasses import dataclass

@dataclass
class User:
    name: str
    age: int

user = User("Anurag", 43)

print(user)
```

48. types

Runtime type utilities.

```
from types import SimpleNamespace

user = SimpleNamespace(
    name="Anurag",
    age=43
)

print(user.name)
```

49. typing

Type annotations.

```
from typing import List

def calculate(values: List[int]) -> int:
    return sum(values)

print(calculate([10, 20, 30]))
```

Modern Python can use:

```
def calculate(values: list[int]) -> int:
    return sum(values)
```

50. abc

Abstract Base Classes.

```
from abc import ABC, abstractmethod

class Storage(ABC):

    @abstractmethod
    def save(self, data):
        pass

class FileStorage(Storage):
```

```
def save(self, data):  
    print("Saving:", data)
```

```
storage = FileStorage()  
storage.save("Hello")
```

Useful for interfaces and plugin architectures.

51. functools

Functional programming utilities.

```
from functools import lru_cache  
  
@lru_cache(maxsize=None)  
def fibonacci(n):  
    if n < 2:  
        return n  
  
    return fibonacci(n - 1) + fibonacci(n - 2)  
  
print(fibonacci(30))
```

52. itertools

Efficient iterator operations.

```
from itertools import combinations

users = ["A", "B", "C"]

for pair in combinations(users, 2):
    print(pair)
```

Excellent for large iterable-processing tasks.

53. operator

Operators as callable functions.

```
from operator import itemgetter

users = [
    ("Anurag", 43),
    ("Sakshi", 30)
]

users.sort(key=itemgetter(1))

print(users)
```

54. contextlib

Create context managers.

```
from contextlib import contextmanager
```

```
@contextmanager
```

```
def transaction():
```

```
    print("BEGIN")
```

```
    try:
```

```
        yield
```

```
        print("COMMIT")
```

```
    except Exception:
```

```
        print("ROLLBACK")
```

```
with transaction():
```

```
    print("Work")
```

55. logging

Production logging.

```
import logging
```

```
logging.basicConfig(level=logging.INFO)
```

```
logging.info("Application started")
```

```
logging.warning("Something needs attention")
```

Prefer this over `print()` for production diagnostics.

56. warnings

Generate warnings.

```
import warnings

warnings.warn(
    "This API is deprecated",
    DeprecationWarning
)
```

57. traceback

Get exception details.

```
import traceback

try:
    10 / 0
except Exception:
    traceback.print_exc()
```

58. pdb

Debug Python code.

```
import pdb
```

```
x = 10  
pdb.set_trace()  
x += 20  
print(x)
```

Modern Python also supports:

```
breakpoint()
```

59. inspect

Inspect functions/classes.

```
import inspect  
  
def add(a, b):  
    return a + b  
  
print(inspect.signature(add))
```

Useful for frameworks and developer tools.

60. dis

Inspect Python bytecode.

```
import dis

def add(a, b):
    return a + b

dis.dis(add)
```

Mostly useful for learning and performance investigation.

61. ast

Parse Python source code.

```
import ast

source = """
x = 10
print(x)
"""

tree = ast.parse(source)

for node in ast.walk(tree):
    print(type(node).__name__)
```

Useful for code analysis and developer tools.

62. compile

Compile source code.

```
source = "print('Hello Python')"  
  
code = compile(  
    source,  
    "<string>",  
    "exec"  
)  
  
exec(code)
```

Never execute untrusted source code.

63. unittest

Unit testing.

```
import unittest  
  
def add(a, b):  
    return a + b  
  
class TestMath(unittest.TestCase):  
  
    def test_add(self):  
        self.assertEqual(add(2, 3), 5)
```

```
if __name__ == "__main__":  
    unittest.main()
```

64. unittest.mock

Mock dependencies.

```
from unittest.mock import Mock  
  
api = Mock()  
  
api.get_user.return_value = {  
    "name": "Anurag"  
}  
  
print(api.get_user())
```

65. doctest

Test examples embedded in documentation.

```
def add(a, b):  
    """  
    >>> add(2, 3)  
    5  
    """  
    return a + b
```

Run:

```
import doctest  
doctest.testmod()
```

66. venv

Create isolated environments.

Command-line usage:

```
python -m venv .venv
```

Activate on Linux/macOS:

```
source .venv/bin/activate
```

Windows:

```
.venv\Scripts\activate
```

67. ensurepip

Bootstrap pip.

```
python -m ensurepip
```

Useful for minimal Python installations.

68. zipapp

Create executable Python ZIP applications.

```
python -m zipapp myapp -m "main:main"
```

Useful for distributing CLI applications.

69. zipfile

Create/read ZIP files.

```
from zipfile import ZipFile  
  
with ZipFile("backup.zip", "w") as zip_file:  
    zip_file.write("data.txt")
```

70. tarfile

Create TAR archives.

```
import tarfile
```

```
with tarfile.open("backup.tar.gz", "w:gz") as archive:  
    archive.add("data")
```

71. gzip

GZIP compression.

```
import gzip  
  
with gzip.open("data.txt.gz", "wt") as f:  
    f.write("Python data")
```

72. bz2

BZIP2 compression.

```
import bz2  
  
compressed = bz2.compress(  
    b"Python standard library"  
)  
  
print(bz2.decompress(compressed))
```

73. lzma

LZMA/XZ compression.

```
import lzma

data = b"Large Python data"

compressed = lzma.compress(data)

print(lzma.decompress(compressed))
```

74. base64

Encode binary data as text.

```
import base64

encoded = base64.b64encode(
    b"Hello Python"
)

print(encoded)

decoded = base64.b64decode(encoded)

print(decoded)
```

Base64 is encoding, not encryption.

75. hashlib

Cryptographic hashes.

```
import hashlib  
  
data = b"Hello Python"  
  
digest = hashlib.sha256(data).hexdigest()  
  
print(digest)
```

Useful for integrity checks.

76. hmac

Message authentication.

```
import hmac  
import hashlib  
  
secret = b"secret"  
message = b"important message"  
  
signature = hmac.new(  
    secret,  
    message,  
    hashlib.sha256  
)hexdigest()  
  
print(signature)
```

Useful for webhook/API signature verification.

77. ssl

TLS/SSL support.

```
import ssl  
  
context = ssl.create_default_context()  
  
print(context)
```

Used with sockets and secure network communication.

78. socket

Low-level networking.

Server:

```
import socket  
  
server = socket.socket()  
  
server.bind(("localhost", 5000))  
server.listen(1)  
  
print("Waiting...")
```

```
connection, address = server.accept()

data = connection.recv(1024)

print(data)

connection.close()
server.close()
```

79. socketserver

Create simple network servers.

```
import socketserver

class Handler(socketserver.BaseRequestHandler):

    def handle(self):
        data = self.request.recv(1024)
        self.request.sendall(data)

server = socketserver.TCPServer(
    ("localhost", 5000),
    Handler
)

server.serve_forever()
```

80. selectors

Multiplex multiple I/O streams.

```
import selectors
import socket

selector = selectors.DefaultSelector()

server = socket.socket()
server.bind(("localhost", 5000))
server.listen()

server.setblocking(False)

selector.register(
    server,
    selectors.EVENT_READ
)

print("Server ready")
```

Useful for event-driven networking.

81. asyncio

Asynchronous programming.

```
import asyncio

async def main():
    print("Start")
```

```
await asyncio.sleep(1)

print("Finished")

asyncio.run(main())
```

Excellent for I/O-heavy applications.

82. threading

Run work in threads.

```
import threading

def worker():
    print("Worker running")

thread = threading.Thread(target=worker)

thread.start()
thread.join()
```

Useful primarily for I/O-bound work.

83. multiprocessing

Process-based parallelism.

```
from multiprocessing import Process

def worker():
    print("Process running")

p = Process(target=worker)

p.start()
p.join()
```

Useful for CPU-heavy work.

84. concurrent.futures

Thread/process pools.

```
from concurrent.futures import ThreadPoolExecutor

def square(x):
    return x * x

with ThreadPoolExecutor(max_workers=4) as executor:
    results = executor.map(square, [1, 2, 3, 4])

print(list(results))
```

85. queue

Thread-safe producer/consumer queue.

```
from queue import Queue

queue = Queue()

queue.put("Job 1")
queue.put("Job 2")

print(queue.get())
print(queue.get())
```

86. subprocess

Run external applications.

```
import subprocess

result = subprocess.run(
    ["python", "--version"],
    capture_output=True,
    text=True
)

print(result.stdout)
```

Avoid unsafe shell commands constructed from user input.

87. signal

Handle OS signals.

```
import signal

def handler(signum, frame):
    print("Shutdown signal received")

signal.signal(signal.SIGINT, handler)

print("Press Ctrl+C")
```

88. mmap

Memory-map a file.

```
import mmap

with open("data.txt", "r+b") as f:
    with mmap.mmap(
        f.fileno(),
        0
    ) as data:
```

```
print(data[:20])
```

Useful with large files.

89. http.client

Low-level HTTP.

```
import http.client

connection = http.client.HTTPSConnection(
    "example.com"
)

connection.request("GET", "/")

response = connection.getresponse()

print(response.status)

connection.close()
```

For most applications, a higher-level HTTP client is easier.

90. http.server

Simple development HTTP server.

```
python -m http.server 8000
```

Or:

```
from http.server import HTTPServer, SimpleHTTPRequestHandler

server = HTTPServer(
    ("localhost", 8000),
    SimpleHTTPRequestHandler
)

server.serve_forever()
```

Not intended as a production web server.

91. urllib.request

Download/access URLs.

```
from urllib.request import urlopen

response = urlopen("https://example.com")

data = response.read()

print(data[:100])
```

92. urllib.parse

Parse URLs.

```
from urllib.parse import urlparse

url = "https://example.com/users?id=10"

result = urlparse(url)

print(result.scheme)
print(result.hostname)
print(result.query)
```

93. urllib.robotparser

Read robots.txt rules.

```
from urllib.robotparser import RobotFileParser

robots = RobotFileParser(
    "https://example.com/robots.txt"
)

robots.read()

print(robots.can_fetch("*", "https://example.com/"))
```

Useful when building crawlers that respect site crawling rules.

94. email

Create email messages.

```
from email.message import EmailMessage

message = EmailMessage()

message["From"] = "sender@example.com"
message["To"] = "receiver@example.com"
message["Subject"] = "Test"

message.set_content("Hello from Python")

print(message)
```

95. smtplib

Send email using SMTP.

```
import smtplib

with smtplib.SMTP("smtp.example.com", 587) as server:
    server.starttls()
    server.login("user", "password")
    server.sendmail(
        "sender@example.com",
        "receiver@example.com",
        "Subject: Test\n\nHello"
    )
```

Use provider-specific authentication/security requirements in production.

96. imaplib

Access an IMAP mailbox.

```
import imaplib

mail = imaplib.IMAP4_SSL(
    "imap.example.com"
)

mail.login(
    "user@example.com",
    "password"
)

mail.select("INBOX")

status, messages = mail.search(
    None,
    "ALL"
)

print(messages)

mail.logout()
```

97. poplib

Access POP3 mail.

```
import poplib

mail = poplib.POP3_SSL(
    "pop.example.com"
)

mail.user("user@example.com")
mail.pass_("password")

print(mail.stat())

mail.quit()
```

98. ftplib

FTP file transfer.

```
from ftplib import FTP

ftp = FTP("ftp.example.com")

ftp.login("username", "password")

print(ftp.nlst())

ftp.quit()
```

Prefer secure alternatives such as SFTP/HTTPS when appropriate.

99. uuid

Generate unique identifiers.

```
import uuid  
request_id = uuid.uuid4()  
print(request_id)
```

Useful for database IDs, request correlation IDs, etc.

100. ipaddress

Work with IP addresses.

```
import ipaddress  
ip = ipaddress.ip_address("192.168.1.10")  
print(ip.version)  
print(ip.is_private)
```

Network example:

```
network = ipaddress.ip_network(  
    "192.168.1.0/24"
```

```
)  
print(network.num_addresses)
```

101. xml.etree.ElementTree

Parse XML.

```
import xml.etree.ElementTree as ET  
  
xml = """  
<user>  
  <name>Anurag</name>  
  <age>43</age>  
</user>  
"""  
  
root = ET.fromstring(xml)  
print(root.find("name").text)
```

102. xml.dom

DOM-style XML processing.

```
from xml.dom import minidom
```

```
xml = """
<users>
  <user>Anurag</user>
</users>
"""

document = minidom.parseString(xml)

users = document.getElementsByTagName("user")

print(users[0].firstChild.data)
```

103. xml.sax

Streaming XML parsing.

```
import xml.sax

class Handler(xml.sax.ContentHandler):

    def characters(self, content):
        if content.strip():
            print(content.strip())

parser = xml.sax.make_parser()

parser.setContentHandler(Handler())

parser.parse("users.xml")
```

Useful for large XML documents.

104. html

Escape HTML.

```
import html

user_input = "<script>alert('test')</script>"

safe = html.escape(user_input)

print(safe)
```

Important when displaying untrusted text in HTML.

105. html.parser

Parse basic HTML.

```
from html.parser import HTMLParser

class Parser(HTMLParser):

    def handle_starttag(self, tag, attrs):
        print("Start:", tag)

parser = Parser()
```

```
parser.feed("""
<html>
<body>
<a href="#">Python</a>
</body>
</html>
""")
```

106. curses

Terminal UI.

```
import curses

def main(screen):
    screen.addstr(5, 5, "Python Terminal UI")
    screen.refresh()
    screen.getch()

curses.wrapper(main)
```

Primarily Unix-like systems.

107. tkinter

Desktop GUI.

```
import tkinter as tk

root = tk.Tk()

label = tk.Label(
    root,
    text="Hello Python"
)

label.pack()

root.mainloop()
```

108. turtle

Educational graphics.

```
import turtle

turtle.forward(100)
turtle.right(90)
turtle.forward(100)

turtle.done()
```

109. gettext

Internationalization.

```
import gettext

translation = gettext.translation(
    "app",
    "locale",
    fallback=True
)

_ = translation.gettext

print(_("Hello"))
```

110. locale

Locale-aware formatting.

```
import locale

locale.setlocale(
    locale.LC_ALL,
    ""
)

print(locale.currency(12345.67))
```

Exact output depends on the configured system locale.

111. argparse

Build command-line interfaces.

```
import argparse

parser = argparse.ArgumentParser()

parser.add_argument(
    "--name",
    required=True
)

args = parser.parse_args()

print("Hello", args.name)
```

Run:

```
python app.py --name Anurag
```

112. getopt

Older/simple command-line parsing.

```
import getopt
import sys
```

```
options, arguments = getopt.getopt(
    sys.argv[1:],
    "n:",
    ["name="]
)

print(options)
```

For new applications, argparse is usually preferable.

113. readline

Interactive command-line editing.

```
import readline

name = input("Name: ")

readline.add_history(name)

print(name)
```

Platform availability varies.

114. cmd

Build an interactive CLI shell.

```
import cmd
```

```
class Shell(cmd.Cmd):  
    prompt = "> "  
  
    def do_hello(self, arg):  
        print("Hello", arg)  
  
    def do_exit(self, arg):  
        return True  
  
Shell().cmdloop()
```

115. logging.config

Configure logging from a dictionary.

```
import logging.config  
import logging  
  
config = {  
    "version": 1,  
    "handlers": {  
        "console": {  
            "class": "logging.StreamHandler"  
        }  
    },  
    "root": {  
        "handlers": ["console"],  
        "level": "INFO"  
    }  
}
```

```
}  
  
logging.config.dictConfig(config)  
  
logging.info("Application started")
```

116. logging.handlers

Rotating log files.

```
import logging  
from logging.handlers import RotatingFileHandler  
  
handler = RotatingFileHandler(  
    "app.log",  
    maxBytes=1_000_000,  
    backupCount=3  
)  
  
logger = logging.getLogger("app")  
  
logger.addHandler(handler)  
logger.setLevel(logging.INFO)  
  
logger.info("Application started")
```

117. contextvars

Context-local variables.

```
import contextvars

request_id = contextvars.ContextVar(
    "request_id"
)

request_id.set("REQ-1001")

print(request_id.get())
```

Especially useful with asynchronous applications.

118. atexit

Run cleanup during normal interpreter shutdown.

```
import atexit

def cleanup():
    print("Cleaning up")

atexit.register(cleanup)

print("Application running")
```

119. resource

Unix resource information.

```
import resource

usage = resource.getrusage(
    resource.RUSAGE_SELF
)

print("User CPU:", usage.ru_utime)
print("System CPU:", usage.ru_stime)
```

Primarily Unix-specific.

120. platform

System information.

```
import platform

print("OS:", platform.system())
print("Release:", platform.release())
print("Python:", platform.python_version())
```

121. sysconfig

Python installation configuration.

```
import sysconfig  
  
print(sysconfig.get_paths())  
print(sysconfig.get_python_version())
```

Useful for packaging/build tools.

122. importlib

Dynamic imports.

```
import importlib  
  
module = importlib.import_module("math")  
  
print(module.sqrt(25))
```

Useful for plugin architectures.

123. pkgutil

Discover packages/modules.

```
import pkgutil  
  
for module in pkgutil.iter_modules():  
    print(module.name)
```

Avoid scanning everything unnecessarily because it can be expensive.

124. pydoc

Generate/display documentation.

```
import pydoc

print(
    pydoc.render_doc("math")
)
```

Command line:

```
python -m pydoc math
```

125. trace

Trace program execution.

```
import trace

tracer = trace.Trace(
    trace=True
)

tracer.runfunc(
    print,
    "Hello Python"
)
```

Mostly a debugging/analysis tool.

126. tracemalloc

Find memory allocations.

```
import tracemalloc
tracemalloc.start()

data = [x for x in range(100000)]

snapshot = tracemalloc.take_snapshot()

for stat in snapshot.statistics("lineno")[:5]:
    print(stat)
```

127. profile

Profile Python code.

```
import profile

def work():
    sum(range(1_000_000))

profile.run("work()")
```

128. cProfile

Detailed performance profiling.

```
import cProfile

def work():
    sum(range(1_000_000))

cProfile.run("work()")
```

Use this to find CPU-intensive functions.

129. graphlib

Python 3.9+ dependency ordering.

```
from graphlib import TopologicalSorter

graph = {
    "database": set(),
    "backend": {"database"},
    "frontend": {"backend"}
}

order = TopologicalSorter(graph).static_order()

print(list(order))
```

Useful for build/dependency systems.

130. zipimport

Import Python modules from ZIP archives.

```
import zipimport

loader = zipimport.zipimporter(
    "application.zip"
)

module = loader.load_module("main")

print(module)
```

For modern packaging, normal packaging/import mechanisms are generally preferable.

131. compression.zstd

Python 3.14+ Zstandard compression facilities.

The exact API should be checked against the Python 3.14 installation because this is a newly added standard-library area.

Conceptually:

```
# Python 3.14+
from compression import zstd
```

```
data = b"Python standard library"
```

```
# Use the zstd compression/decompression  
# APIs provided by the installed Python 3.14 version.
```

This is intended for high-performance modern compression.

132. annotationlib

Python 3.14+ annotation introspection.

```
# Python 3.14+
```

```
import annotationlib  
  
def greet(name: str) -> str:  
    return "Hello " + name  
  
annotations = annotationlib.get_annotations(  
    greet  
)  
  
print(annotations)
```

Primarily useful to framework/library developers working with Python type annotations.

Historical Python libraries

The following libraries from the previous answer are particularly important because they demonstrate how Python's standard library evolved.

133. sets

Historical Python 2 library.

Old-style example:

```
from sets import Set
numbers = Set([1, 2, 3, 3])
print(numbers)
```

Modern replacement:

```
numbers = {1, 2, 3}
```

sets should **not** be used in modern Python.

134. cStringIO

Historical Python 2 module.

Old:

```
from cStringIO import StringIO  
buffer = StringIO()  
buffer.write("Hello")  
print(buffer.getvalue())
```

Modern replacement:

```
from io import StringIO  
buffer = StringIO()  
buffer.write("Hello")  
print(buffer.getvalue())
```

135. StringIO

Historical module.

Modern equivalent:

```
from io import StringIO  
stream = StringIO()  
stream.write("Python")  
print(stream.getvalue())
```

136. md5

Historical hashing module.

Old:

```
import md5

digest = md5.new("hello").hexdigest()

print(digest)
```

Modern:

```
import hashlib

digest = hashlib.md5(
    b"hello"
).hexdigest()

print(digest)
```

MD5 should generally **not** be used for security-sensitive hashing.

137. sha

Historical SHA module.

Old:

```
import sha

digest = sha.new("hello").hexdigest()

print(digest)
```

Modern:

```
import hashlib

digest = hashlib.sha256(
    b"hello"
).hexdigest()

print(digest)
```

138. ConfigParser

Historical Python 2 capitalization.

Old:

```
import ConfigParser

config = ConfigParser.ConfigParser()
```

Modern:

```
import configparser

config = configparser.ConfigParser()
```

139. Queue

Historical Python 2 module.

Old:

```
import Queue
queue = Queue.Queue()
queue.put("Job")
print(queue.get())
```

Modern:

```
from queue import Queue
queue = Queue()
queue.put("Job")
print(queue.get())
```

140. SimpleHTTPServer

Historical Python 2 module.

Old:

```
python -m SimpleHTTPServer 8000
```

Modern:

```
python -m http.server 8000
```

141. BaseHTTPServer

Historical Python 2 HTTP server module.

Old:

```
from BaseHTTPServer import HTTPServer
```

Modern:

```
from http.server import HTTPServer
```

142. SocketServer

Historical Python 2 module.

Old:

```
import SocketServer
```

Modern:

```
import socketserver
```

143. Tkinter

Historical Python 2 name.

Old:

```
import Tkinter
```

Modern:

```
import tkinter  
root = tkinter.Tk()  
root.mainloop()
```

144. cPickle

Historical optimized pickle implementation.

Old:

```
import cPickle
```

```
data = cPickle.dumps(  
    {"name": "Anurag"}  
)
```

Modern:

```
import pickle  
  
data = pickle.dumps(  
    {"name": "Anurag"}  
)
```

145. imp

Historical import API.

Old:

```
import imp  
  
module = imp.load_source(  
    "mymodule",  
    "mymodule.py"  
)
```

Modern approach:

```
import importlib  
  
module = importlib.import_module(  
    "mymodule"
```

```
)
```

146. optparse

Older command-line parser.

```
from optparse import OptionParser

parser = OptionParser()

parser.add_option(
    "-n",
    "--name"
)

options, args = parser.parse_args()

print(options.name)
```

For new programs, use argparse.

147. distutils

Historical Python packaging/build module.

Old:

```
from distutils.core import setup
```

```
setup(  
    name="myapp",  
    version="1.0"  
)
```

distutils was removed from modern Python. Modern Python packaging uses tools such as setuptools, build, and pip.

148. asyncore

Historical asynchronous networking module.

Old:

```
import asyncore  
asyncore.loop()
```

Modern approach:

```
import asyncio  
asyncio.run(main())
```

Use asyncio for modern asynchronous applications.

149. asynchat

Historical asynchronous chat protocol helper.

Old:

```
import asynchat
```

Modern applications generally use:

```
import asyncio
```

For example:

```
async def handle():
    reader, writer = await asyncio.open_connection(
        "localhost",
        8000
    )

    writer.write(b"Hello")
    await writer.drain()

    writer.close()
```

150. cgi

Historical CGI support.

Old:

```
import cgi

form = cgi.FieldStorage()
```

```
print(form)
```

Modern web applications generally use frameworks/servers based on WSGI or ASGI rather than the old CGI module.

151. telnetlib

Historical Telnet client.

Old:

```
import telnetlib

connection = telnetlib.Telnet(
    "example.com",
    23
)

connection.close()
```

Telnet is insecure because it does not provide encryption. Prefer SSH or another secure protocol when appropriate.

152. aifc

Historical AIFF audio-file support.

Old:

```
import aifc
```

```
audio = aifc.open(
    "sound.aiff",
    "rb"
)

print(audio.getnchannels())

audio.close()
```

This module has been removed from current Python. Modern applications generally use dedicated audio libraries.

153. audioop

Historical raw-audio processing.

Old:

```
import audioop

result = audioop.rms(
    b"\x00\x01",
    1
)

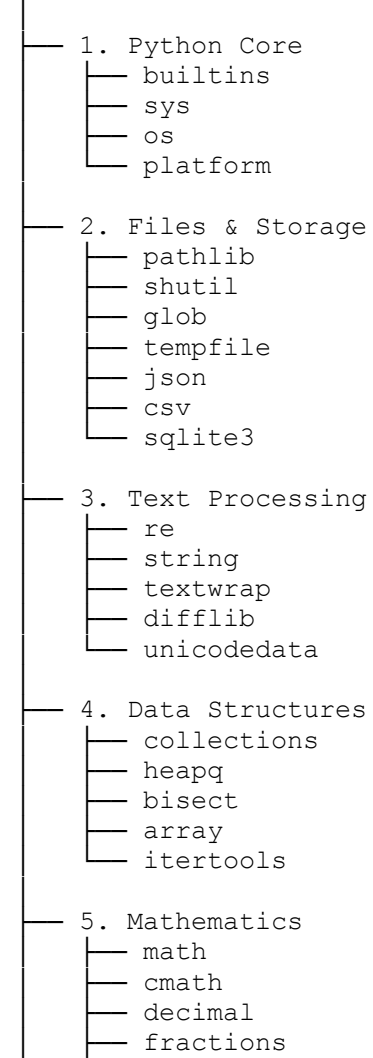
print(result)
```

It was removed from modern Python. Use an appropriate third-party audio/numerical package when audio processing is required.

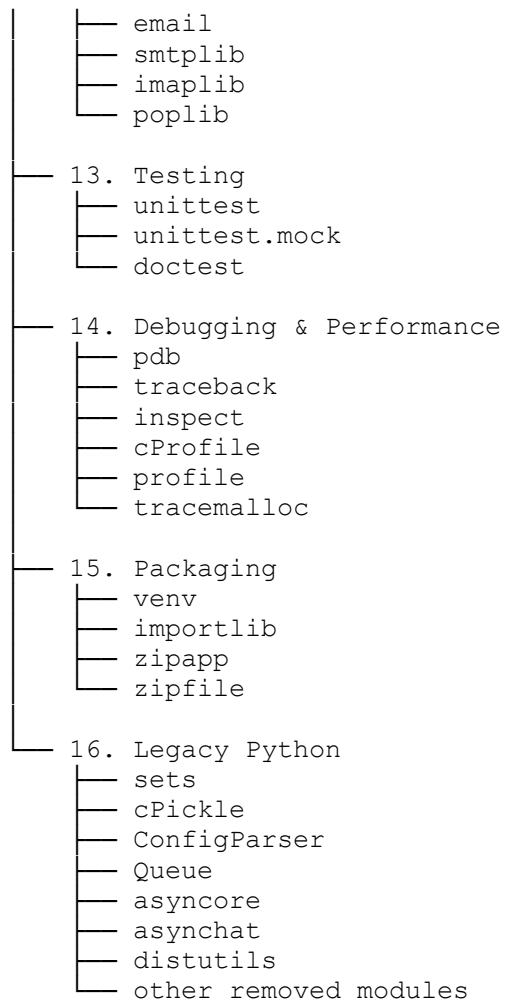
A useful way to learn all of these

Rather than memorizing 150+ modules individually, organize them into a **Python Standard Library learning path**:

Python Standard Library



	└─ statistics
6. Date & Time	
	└─ datetime
	└─ time
	└─ calendar
	└─ zoneinfo
7. OOP & Type System	
	└─ dataclasses
	└─ enum
	└─ typing
	└─ types
	└─ abc
8. Functional Programming	
	└─ functools
	└─ itertools
	└─ operator
9. Networking	
	└─ socket
	└─ ssl
	└─ urllib
	└─ http
	└─ ipaddress
10. Concurrency	
	└─ threading
	└─ multiprocessing
	└─ asyncio
	└─ queue
	└─ concurrent.futures
11. Security	
	└─ secrets
	└─ hashlib
	└─ hmac
	└─ ssl
12. Email	



Most important modules for modern Python development

If your goal is **Java/Spring Boot + Python/FastAPI/backend/AI development**, I would prioritize:

Level 1

- os
- pathlib
- sys
- json
- re
- datetime
- collections
- logging

Level 2

- typing
- dataclasses
- functools
- itertools
- sqlite3
- csv
- configparser

Level 3

- asyncio
- threading
- multiprocessing
- concurrent.futures
- subprocess
- socket

Level 4

- hashlib
- secrets
- hmac
- ssl

└─ ipaddress

Level 5

├─ unittest
├─ unittest.mock
├─ pdb
├─ inspect
├─ cProfile
└─ tracemalloc

Level 6

├─ ast
├─ importlib
├─ contextlib
├─ contextvars
├─ graphlib
└─ annotationlib

This gives you a much more practical progression than trying to memorize the entire standard library alphabetically.

Below is a consolidated list of the **most commonly used Python standard-library modules across the Python 1.0 → 3.14 era**, emphasizing libraries that have had significant practical use in real applications. It is not a list of every module ever shipped with Python; many historical modules were obscure, platform-specific, renamed, or removed.

#	Library / Module	Main Purpose	Common Real-Time Use Case	Typical Application
1	os	Operating-system interaction	Read environment variables, create directories, execute OS operations	Backend, DevOps, automation
2	sys	Python runtime/interpreter	CLI arguments, interpreter information, exit handling	CLI tools, scripts
3	pathlib	Filesystem paths	Create/read/search files and directories	Backend, automation
4	os.path	Path manipulation	Join/check file paths	Legacy applications
5	shutil	High-level file operations	Copy/move/delete/backup files	Backup, deployment
6	glob	File pattern matching	Find all .csv, .log, .jpg files	Data processing
7	tempfile	Temporary files	Process uploaded files temporarily	Web applications
8	io	Stream/file-like objects	Process data in memory	APIs, file processing
9	json	JSON serialization	REST API request/response processing	Web/API development
10	csv	CSV processing	Import/export spreadsheet data	Data processing
11	sqlite3	Embedded SQL database	Store application data locally	Desktop apps, prototypes
12	pickle	Python object serialization	Cache/save Python objects	Internal Python applications
13	configparser	INI configuration	Read application settings	Legacy/backend apps

14	tomllib	TOML parsing	Read pyproject.toml/configuration	Modern Python tooling
15	re	Regular expressions	Extract/validate emails, phone numbers, IDs	Data validation
16	string	String utilities	Character sets, formatting helpers	Text processing
17	textwrap	Text formatting	Format terminal/report output	CLI/reporting
18	difflib	Difference comparison	Compare configuration/source files	Developer tools
19	unicodedata	Unicode processing	Normalize international text	Search/text systems
20	math	Mathematics	Engineering/business calculations	General applications
21	decimal	Precise decimal arithmetic	Currency calculations	Banking/finance
22	fractions	Exact fractions	Ratio/fraction calculations	Mathematics/education
23	statistics	Statistical calculations	Average/median/variance	Analytics
24	random	Pseudo-random numbers	Simulations/test data	Testing/simulation
25	secrets	Secure random values	Authentication/reset tokens	Security/web applications
26	datetime	Date/time handling	Calculate deadlines, timestamps	Business applications
27	zoneinfo	Time zones	Convert India/US/Europe times	Global applications
28	time	Low-level time	Sleep/timers/performance timing	Systems programming
29	calendar	Calendar operations	Generate monthly/date information	Scheduling
30	collections	Specialized containers	Count/group/manage data	Backend/data processing

31	collections.abc	Collection interfaces	Check iterable/sequence interfaces	Framework development
32	heapq	Heap/priority queue	Prioritize jobs/tasks	Scheduling
33	bisect	Sorted-list operations	Maintain ranked/sorted data	Algorithms
34	array	Compact typed arrays	Memory-efficient numeric storage	Systems/data processing
35	copy	Object copying	Clone configuration/data structures	Application logic
36	weakref	Weak object references	Memory-sensitive caches	Frameworks/caching
37	enum	Enumerations	Status/type constants	Enterprise applications
38	dataclasses	Data classes	DTO/request/response models	APIs/backend
39	typing	Type annotations	Type-safe function/API definitions	Modern Python
40	types	Runtime type utilities	Dynamic namespaces/type operations	Frameworks
41	abc	Abstract classes/interfaces	Define service/repository contracts	Enterprise architecture
42	functools	Functional utilities	Caching/decorators/partial functions	Backend/algorithms
43	itertools	Iterator operations	Efficient large-data processing	Data engineering
44	operator	Operators as functions	Sorting/grouping by fields	Data processing
45	contextlib	Context managers	Resource/transaction management	Backend
46	logging	Application logging	Record errors/events	Production systems
47	warnings	Runtime warnings	Deprecation warnings	Libraries/frameworks

48	traceback	Exception details	Record stack traces	Error monitoring
49	pdb	Debugging	Debug application code	Development
50	inspect	Introspection	Inspect function signatures/classes	Frameworks/tools
51	ast	Python syntax-tree processing	Analyze Python source code	Linters/code tools
52	dis	Bytecode inspection	Investigate Python execution	Performance/education
53	unittest	Unit testing	Automated tests	Software development
54	unittest.mock	Mocking	Mock APIs/database/services	Unit testing
55	doctest	Documentation tests	Test examples in docs	Libraries/education
56	timeit	Micro-benchmarking	Compare algorithm performance	Optimization
57	cProfile	CPU profiling	Find slow functions	Performance engineering
58	profile	Python profiling	Analyze execution	Performance engineering
59	tracemalloc	Memory profiling	Find memory allocation problems	Backend/performance
60	venv	Virtual environments	Isolate project dependencies	Python development
61	importlib	Dynamic importing	Load plugins/modules	Frameworks/plugins
62	pkgutil	Module discovery	Discover installed modules/plugins	Tooling
63	zipfile	ZIP archives	Package/backup files	Deployment/backup
64	tarfile	TAR archives	Linux/server backups	DevOps

65	gzip	GZIP compression	Compress logs/data	Backend/DevOps
66	bz2	BZIP2 compression	Compress archival data	Data processing
67	lzma	LZMA/XZ compression	High-ratio compression	Backup/archival
68	base64	Binary-to-text encoding	Encode API/file payloads	Web/API
69	hashlib	Cryptographic hashes	File integrity/checksums	Security
70	hmac	Message authentication	Verify API/webhook signatures	Security
71	ssl	TLS/SSL	Secure network connections	Networking
72	socket	TCP/UDP networking	Build network clients/servers	Network programming
73	selectors	I/O multiplexing	Handle many sockets efficiently	Network servers
74	asyncio	Asynchronous programming	Handle many concurrent I/O tasks	APIs, networking
75	threading	Thread concurrency	Parallel I/O operations	Automation/backend
76	multiprocessing	Process concurrency	CPU-heavy parallel processing	Data/AI processing
77	concurrent.futures	Thread/process pools	Execute many tasks concurrently	Backend/data processing
78	queue	Thread-safe queues	Producer/consumer workers	Background processing
79	subprocess	External processes	Run Git, OS, CLI applications	DevOps/automation
80	signal	OS signal handling	Graceful service shutdown	Server applications
81	mmap	Memory-mapped files	Process very large files	Data processing

82	urllib.request	HTTP/URL requests	Download/access web resources	Simple API clients
83	urllib.parse	URL parsing	Parse query strings/URLs	Web applications
84	http.client	Low-level HTTP	Custom HTTP communication	Network programming
85	http.server	Basic HTTP server	Local development/file serving	Testing/development
86	email	Email message processing	Build/parse MIME email	Email automation
87	smtplib	SMTP	Send emails	Notifications
88	imaplib	IMAP	Read/manage inboxes	Email automation
89	poplib	POP3	Retrieve email	Legacy email systems
90	ftplib	FTP	Legacy file transfer	Enterprise/legacy systems
91	uuid	Unique IDs	Generate request/database IDs	Distributed systems
92	ipaddress	IP address operations	Validate/classify IPs	Networking/security
93	xml.etree.ElementTree	XML processing	Parse XML configuration/API data	Enterprise integration
94	xml.dom	XML DOM processing	Manipulate XML documents	XML applications
95	xml.sax	Streaming XML parsing	Process huge XML files	Data integration
96	html	HTML escaping	Sanitize displayed text	Web applications
97	html.parser	Basic HTML parsing	Extract information from HTML	Lightweight parsing

98	argparse	CLI arguments	Build professional CLI applications	DevOps/tools
99	gettext	Translation/i18n	Multilingual applications	Enterprise applications
100	locale	Locale-specific formatting	Currency/number/date formatting	International applications
101	platform	Platform information	Detect OS/Python environment	Deployment/diagnostics
102	sysconfig	Python installation configuration	Build/package tooling	Python development
103	contextvars	Context-local state	Request IDs in async services	Web/backend
104	graphlib	Graph algorithms	Dependency ordering	Build systems/workflows
105	tkinter	Desktop GUI	Create desktop applications	Desktop software
106	curses	Terminal UI	Interactive terminal dashboards	Linux/Unix tools
107	turtle	Graphics/education	Teach programming/geometry	Education
108	pydoc	Documentation	Generate/display module documentation	Developer tools
109	readline	Interactive terminal editing	Command history/autocomplete	CLI applications
110	cmd	Interactive command shell	Build custom CLI shells	Administration tools
111	zipapp	Executable ZIP applications	Package Python CLI programs	Deployment
112	ensurepip	Bootstrap pip	Install/repair pip	Python environment setup

113	graphlib	Dependency graph processing	Resolve task dependencies	Build/workflow systems
114	annotationlib	Annotation inspection	Framework/type annotation processing	Python 3.14+ tooling
115	compression.zstd	Zstandard compression	Fast modern compression	Python 3.14+ data processing

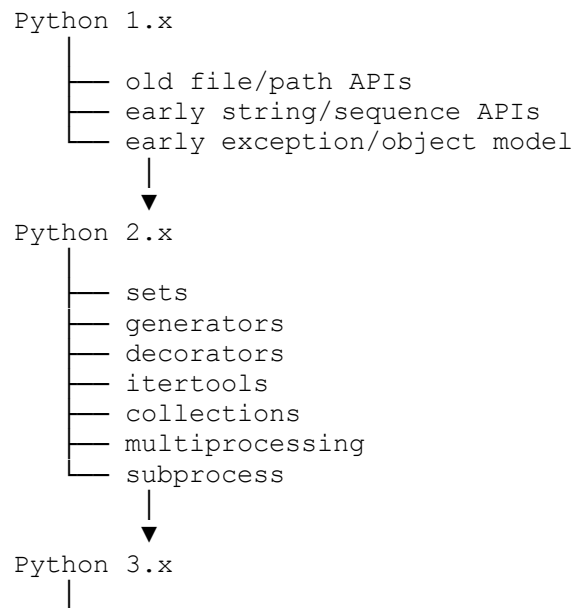
Highest-value libraries by real-world domain

Domain	Most useful standard libraries
Web/API Backend	os, pathlib, json, datetime, logging, typing, dataclasses, asyncio, urllib, ssl
FastAPI/REST development	asyncio, json, typing, dataclasses, logging, datetime, uuid, secrets
AI/ML applications	json, pathlib, csv, pickle, sqlite3, multiprocessing, concurrent.futures, logging
Cybersecurity	hashlib, hmac, secrets, ssl, socket, ipaddress, re, logging, subprocess
Data engineering	csv, json, sqlite3, itertools, collections, gzip, bz2, lzma, multiprocessing
Automation	os, pathlib, shutil, glob, subprocess, datetime, logging, argparse
DevOps	os, sys, subprocess, pathlib, json, logging, argparse, tarfile, zipfile
Email automation	email, smtplib, imaplib, poplib, re, datetime
Networking	socket, ssl, selectors, asyncio, urllib, ipaddress
Desktop applications	tkinter, sqlite3, pathlib, logging, json

CLI applications	argparse, sys, pathlib, logging, cmd, readline
Testing	unittest, unittest.mock, doctest, timeit
Performance	timeit, cProfile, profile, tracemalloc, multiprocessing, itertools
Security	secrets, hashlib, hmac, ssl, ipaddress
File processing	pathlib, os, shutil, glob, csv, json, zipfile, tarfile, gzip
Internationalization	gettext, locale, unicodedata, zoneinfo

Historical → modern equivalents

For learning Python **1.0 through 3.14**, these transitions are particularly important:



- |— pathlib
- |— asyncio
- |— typing
- |— dataclasses
- |— f-strings
- |— zoneinfo
- |— match/case
- |— Exception Groups
- |— modern type parameters
- |— free-threading
- |— 3.14 additions

For **modern Python 3.14 development**, the most important distinction is that many libraries from Python 1.x/2.x are primarily of **historical interest**. The current Python standard library is documented by Python's official Library Reference and Module Index. [Python 3.14 Standard Library](#) [Python 3.14 Module Index](#)